Project Report Format :

ShopEZ – One Stop Shop for Online Purchases

BY:

Team ID : LTVIP2026TMIDS41352

Team Size : 4

Team Leader : Vuyyuru Sri Dinesh

Team member : Velaga Harshavardhan

Team member : Chilukuri Purna Chandra Reddy

Team member : Amaladasu Sai Ram

---

1. INTRODUCTION

1.1 Project Overview

ShopEZ is a full-stack e-commerce web application developed using the MERN stack, which consists of MongoDB, Express.js, React.js, and Node.js. The platform is designed to provide a seamless, secure, and efficient online shopping experience for customers while offering powerful order and inventory management tools for sellers and administrators.

The system enables customers to browse products, apply advanced filtering options, add products to their cart, complete secure checkout, and track their orders. At the same time, sellers can manage product listings and orders through a dedicated dashboard, and administrators can monitor system activities and manage users.

ShopEZ integrates frontend user experience with robust backend logic and database management to deliver a scalable and maintainable solution.

1.2 Purpose

The primary purpose of this project is to design and implement a user-friendly and secure e-commerce platform that addresses common challenges faced by online shoppers and sellers.

The specific objectives include:

- Simplifying product discovery through smart filtering
- Reducing cart abandonment with seamless checkout
- Ensuring secure authentication using JWT
- Providing sellers with efficient order management tools
- Building a scalable architecture using modern web technologies

---

2. IDEATION PHASE

2.1 Problem Statement

In today's fast-paced world, users often struggle with cluttered e-commerce platforms that make product discovery time-consuming and overwhelming. Complicated checkout processes, lack of personalization, and security concerns further reduce user satisfaction. Additionally, small and medium sellers face difficulties in managing orders and tracking sales due to limited dashboard functionalities.

Therefore, there is a need for a streamlined, secure, and scalable e-commerce platform that enhances user experience while empowering sellers with efficient management tools.

2.2 Empathy Map Canvas

To better understand the target users, an empathy map was created based on a user persona named Sarah, a busy working professional.

Thinks:

- Is this product suitable for my needs?
- Is my payment information secure?
- Will the delivery arrive on time?

Feels:

- Overwhelmed by too many product options
- Pressured due to limited time
- Relieved after successful checkout

Says:

- I need something simple and fast.
- I want secure transactions.

Does:

- Uses filters to narrow search
- Compares product prices
- Reads reviews before purchasing

Insight gained: Users value simplicity, speed, trust, and clarity in online shopping platforms.

2.3 Brainstorming

During the brainstorming phase, the team generated multiple ideas without restriction. These ideas were later grouped and prioritized.

Generated Ideas:

- Smart filtering system
- Role-based authentication
- Seller dashboard
- Order tracking system
- Secure checkout
- Reviews and ratings
- Admin monitoring panel
- Payment gateway integration

After evaluation, core features were prioritized based on impact and feasibility.

---

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey includes the following stages:

1. Visiting the website
2. Browsing products
3. Applying filters
4. Viewing product details
5. Adding items to cart
6. Completing checkout
7. Receiving order confirmation
8. Tracking order

Pain points addressed:

- Slow search process
- Confusing navigation
- Security concerns

3.2 Solution Requirements

Functional Requirements:

- User Registration & Login
- Product Management
- Cart Management
- Checkout System
- Order Processing
- Seller Dashboard
- Admin Dashboard

Non-Functional Requirements:

- Security (JWT & bcrypt)
- Scalability
- High Availability
- Performance Optimization
- Maintainability

3.3 Data Flow Diagram

Level 0 DFD:

Customer → ShopEZ System → Database

Level 1 DFD:

Authentication Module → Product Module → Cart Module → Order Module → MongoDB

Database Collections:

- Users
- Products
- Orders
- Cart

3.4 Technology Stack

Frontend:

- React.js
- HTML5
- CSS3
- Bootstrap

Backend:

- Node.js
- Express.js

Database:

- MongoDB

Security:

- JSON Web Tokens (JWT)
- bcrypt password hashing

Deployment:

- MongoDB Atlas
- Render / Vercel

---

4. PROJECT DESIGN

4.1 Problem–Solution Fit

Problem:
Users experience difficulty finding products quickly and completing checkout securely.

Solution:
ShopEZ provides intelligent filtering, secure login, streamlined checkout, and seller management dashboards.

Outcome:
Improved user satisfaction and reduced cart abandonment rates.

4.2 Proposed Solution

ShopEZ is a scalable MERN-based e-commerce application that integrates a clean UI, secure backend services, and efficient database operations.

Revenue Model:

- Commission per sale
- Premium seller subscription
- Featured product promotions

4.3 Solution Architecture

The system follows a three-tier architecture:

Presentation Layer – React frontend
Application Layer – Node + Express backend
Data Layer – MongoDB database

External services include:

- Payment gateway API
- Email notification service

This architecture ensures modularity, scalability, and security.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project followed Agile methodology with multiple sprints.

Sprint 1:

- Authentication
- Product Listing

Sprint 2:

- Cart & Checkout

Sprint 3:

- Seller Dashboard

Average Velocity:
13 Story Points per Sprint

---

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Functional testing covered:

- Registration
- Login
- Cart operations
- Checkout process
- Dashboard access

Performance metrics:

- Page load time under 3 seconds

- Optimized API response time
- Indexed MongoDB queries

No critical bugs remained before release.

---

7. RESULTS

7.1 Output Screenshots

- Login Page
- Product Listing Page
- Cart Page
- Checkout Page
- Seller Dashboard
- Admin Dashboard

All major functionalities were successfully implemented and tested.

---

8. ADVANTAGES & DISADVANTAGES

Advantages:

- Secure authentication
- User-friendly interface
- Scalable architecture
- Modular structure

Disadvantages:

- Limited AI recommendation features
- Payment integration requires production setup
- Initial deployment configuration complexity

---

9. CONCLUSION

ShopEZ successfully addresses common online shopping challenges by delivering a secure, scalable, and user-friendly e-commerce platform. The project demonstrates strong integration between frontend, backend, and database technologies while maintaining system security and performance.

The system is capable of further expansion and real-world deployment with minimal modifications.

---

10. FUTURE SCOPE

- AI-powered product recommendation system
- Chatbot integration
- Mobile application version
- Real-time order tracking
- Microservices-based architecture
- Advanced analytics dashboard

---

11. APPENDIX

Source Code:

This includes:

- Backend (Node + Express + MongoDB)
- Frontend (React)
- JWT Authentication
- Product Listing
- Add to Cart (basic)

📁 **Project Structure**

```
shopez/
├── server/
└── client/
```

🚀 **BACKEND (server)**

1️⃣ Initialize Backend

```
mkdir server
cd server
npm init -y
npm install express mongoose dotenv cors bcryptjs jsonwebtoken
```

📄 server/server.js

```
const express = require("express");
const mongoose = require("mongoose");
const dotenv = require("dotenv");
const cors = require("cors");

dotenv.config();
const app = express();

app.use(express.json());
app.use(cors());

mongoose.connect(process.env.MONGO_URI)
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

app.use("/api/users", require("./routes/userRoutes"));
app.use("/api/products", require("./routes/productRoutes"));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

📄 server/.env

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/shopez
JWT_SECRET=shopezsecret
```

📂 Models

📄 models/User.js

```
const mongoose = require("mongoose");

const userSchema = mongoose.Schema({
  name: String,
  email: { type: String, unique: true },
  password: String,
  isAdmin: { type: Boolean, default: false }
});

module.exports = mongoose.model("User", userSchema);
```

📄 **models/Product.js**

```
const mongoose = require("mongoose");

const productSchema = mongoose.Schema({
  name: String,
  price: Number,
  description: String,
  image: String,
  countInStock: Number
});

module.exports = mongoose.model("Product", productSchema);
```

📂 **Routes**

📄 **routes/userRoutes.js**

```
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const router = express.Router();

// Register
router.post("/register", async (req, res) => {
  const { name, email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);
```

```
  const user = await User.create({
    name,
    email,
    password: hashedPassword
  });

  res.json({ message: "User Registered" });
});

// Login
router.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (user && await bcrypt.compare(password, user.password)) {
    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET);
    res.json({ token });
  } else {
    res.status(401).json({ message: "Invalid credentials" });
  }
});

module.exports = router;
```

---

routes/productRoutes.js

```
const express = require("express");
const Product = require("../models/Product");

const router = express.Router();

// Get All Products
router.get("/", async (req, res) => {
  const products = await Product.find();
  res.json(products);
});

// Add Product
router.post("/", async (req, res) => {
  const product = await Product.create(req.body);
  res.json(product);
});
```

```
module.exports = router;
```

🚀 FRONTEND (client)

1️⃣ Create React App

```
npx create-react-app client
cd client
npm install axios react-router-dom
```

📄 client/src/App.js

```
import React from "react";
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import Login from "./pages/Login";

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
      </Routes>
    </Router>
  );
}

export default App;
```

📄 client/src/pages/Home.js

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function Home() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
```

```
    axios.get("http://localhost:5000/api/products")
      .then(res => setProducts(res.data));
  }, []);

  return (
    <div>
      <h2>ShopEZ Products</h2>
      {products.map(product => (
        <div key={product._id}>
          <h3>{product.name}</h3>
          <p>₹{product.price}</p>
        </div>
      ))}
    </div>
  );
}

export default Home;
```

---

client/src/pages/Login.js

```
import React, { useState } from "react";
import axios from "axios";

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const loginHandler = async () => {
    const { data } = await axios.post(
      "http://localhost:5000/api/users/login",
      { email, password }
    );
    localStorage.setItem("token", data.token);
    alert("Login Successful");
  };

  return (
    <div>
      <h2>Login</h2>
      <input type="email" onChange={(e) => setEmail(e.target.value)} />
      <input type="password" onChange={(e) => setPassword(e.target.value)} />
      <button onClick={loginHandler}>Login</button>
    </div>
```

```
  );
}

export default Login;
```

▶️ Run the Application

Start Backend

```
cd server
npm start
```

Start Frontend

```
cd client
npm start
```

✅ What This Version Includes

✓ User Registration
✓ User Login (JWT)
✓ MongoDB Connection
✓ Product Listing
✓ Basic React UI

GitHub Repository Link:

https://github.com/sridinesh04/MERNSTACK-_SHOPEZ-SHOP-TO-STOP-.git

Dataset Link:
Not Applicable (Dynamic Database System)

Project Demo Link: https://drive.google.com/file/d/19JD4BLqI-XjL60Da4nQqLJC1GOEyewG7/view?usp=drive_link
Deployment URL:

https://drive.google.com/file/d/19JD4BLqI-XjL60Da4nQqLJC1GOEyewG7/view?usp=drive_link